

JAVA

Complete Notes & Interview Reference

Covering:

- Language Fundamentals
 - Arrays
 - Variables & Var-Args
 - Operators & Assignments
 - Flow Control
 - OOP (Inheritance, Polymorphism, Abstraction, Encapsulation)
 - Interfaces & Modifiers
 - String, StringBuffer & StringBuilder
 - Collections Framework (List, Set, Map, Queue)
 - Exception Handling
 - Multithreading & Concurrency
 - Inner Classes
 - Generics & Type Safety
 - Java 8 Streams & Lambdas
 - Garbage Collection & JVM
-

 **With Interview Cheatsheets for Every Topic**

1. Language Fundamentals

Identifiers & Naming Rules

- ▶ An identifier is a name given to a variable, method, class, interface, or package.
- ▶ Allowed characters: alphanumeric [A-Z, a-z, 0-9], \$ (dollar), _ (underscore). Cannot start with a digit.
- ▶ Java is case-sensitive. No length limit — but 4–15 letters is advisable.
- ▶ Keywords/reserved words (e.g., int, class, static) cannot be used as identifiers.
- ▶ Predefined class names can be used as identifiers (e.g., int String = 10; is valid but discouraged).
- ▶ Java has 71 reserved words: 51 keywords, 3 reserved literals (true, false, null), 17 contextual keywords.
- ▶ null is the default value for all uninitialized object reference variables.

Primitive Data Types

Java has 8 primitive data types — all signed (except char):

Type	Size	Range / Notes
byte	8 bits (1 byte)	-128 to 127
short	16 bits (2 bytes)	-32,768 to 32,767
int	32 bits (4 bytes)	-2,147,483,648 to 2,147,483,647 (default for integral literals)
long	64 bits (8 bytes)	Use suffix L or l
float	32 bits (4 bytes)	Use suffix f or F (memory-efficient)
double	64 bits (8 bytes)	Default for floating-point literals; preferred for precision
char	16 bits (2 bytes)	0 to 65,535 (Unicode); char → int promotion allowed
boolean	JVM-dependent	true or false only

- ▶ byte is ideal for data streams (files/networks work in bytes).
- ▶ float for memory efficiency; double for scientific/financial precision.
- ▶ Java is Unicode-based, using the Unicode standard for characters.

Wrapper Classes

Wrapper classes convert primitives into objects, required for collections and APIs.

Primitive	Wrapper	Key Note
byte	Byte	Cached: -128 to 127
short	Short	Cached: -128 to 127
int	Integer	Cached: -128 to 127 (configurable)
long	Long	Cached: -128 to 127

Primitive	Wrapper	Key Note
float	Float	No cache
double	Double	No cache
char	Character	Cached: 0 to 127
boolean	Boolean	Always cached (TRUE / FALSE)

Literals

- ▶ Integral literals: Decimal (10), Octal (012), Hexadecimal (0xA), Binary (0b1010).
- ▶ Default integral literal type is int; use L/l suffix for long.
- ▶ Default floating-point literal type is double; use f/F suffix for float.
- ▶ char literal: single character in single quotes — char ch = 'a'; also as Unicode \u0061.
- ▶ String: any sequence of characters in double quotes.
- ▶ Underscore in numeric literals (Java 7+): int x = 1_000_000; — not at start/end or next to decimal.
- ▶ Escape sequences: \\ (backslash), \' (single quote), \" (double quote), \n, \r, \t, \b, \f.

Default Values

Type	Default Value
byte / short / int / long	0
float	0.0f
double	0.0d
char	\u0000 (null character)
boolean	false
Object reference	null

Autoboxing & Unboxing (Java 5+)

- ▶ Autoboxing: automatic conversion of primitive → wrapper (Integer i = 10; → Integer.valueOf(10)).
- ▶ Unboxing: automatic conversion of wrapper → primitive (int x = i; → i.intValue()).
- ▶ NullPointerException possible if wrapper reference is null during unboxing.
- ▶ Widening > Autoboxing > Var-args in method resolution priority.

JavaBean Standard

- ▶ Must have a no-argument constructor.
- ▶ Properties should be private with public getters and setters.
- ▶ For boolean properties: use is prefix (isEmpty(), isValid()).
- ▶ Implements Serializable (optional but recommended).

Coding Conventions

Component	Convention	Example
Class	PascalCase (noun)	StudentRecord, BankAccount
Interface	PascalCase (noun/capability)	Comparable, Serializable
Method	camelCase (verb-noun)	calculateTotal(), getUsername()
Variable	camelCase (noun)	studentName, totalBalance
Constant	UPPER_SNAKE_CASE	MAX_SIZE, DEFAULT_TIMEOUT
Package	all lowercase	com.example.myapp



Interview Cheatsheet — Language Fundamentals

Method / Concept	Class	Purpose
parseInt(String)	Integer	Parse string to int; throws NumberFormatException
valueOf(String)	Integer / Double etc.	Create wrapper object from string
intValue() / doubleValue()	Wrapper classes	Extract primitive from wrapper
toBinaryString(int)	Integer	Convert int to binary string
toHexString(int)	Integer	Convert int to hex string
toOctalString(int)	Integer	Convert int to octal string
Integer.MAX_VALUE / MIN_VALUE	Integer	2147483647 / -2147483648
Character.isLetter(c)	Character	Check if char is a letter
Character.isDigit(c)	Character	Check if char is a digit
Character.toLowerCase(c)	Character	Convert char to lowercase
Character.toUpperCase(c)	Character	Convert char to uppercase
Objects.isNull(obj)	Objects	Null check (Java 8+)
System.exit(0)	System	Terminate JVM (prevents finally block)
Runtime.getRuntime().gc()	Runtime	Request garbage collection

2. Arrays

Declaration & Initialization

- ▶ An array is an indexed, fixed-size collection of homogeneous elements.

```
int[] arr;           // Declaration (size not specified)
arr = new int[5];     // Initialization with size
int[] arr2 = {1,2,3}; // Declaration + initialization
int[][] matrix = new int[3][3]; // 2D array
```

Key Rules

- ▶ Size cannot be changed after array creation.
- ▶ Size must be specified during initialization, not declaration.
- ▶ Valid size types: byte, short, char, int — not long, float, double.
- ▶ Negative size → `NegativeArraySizeException` at runtime.
- ▶ Zero-size array is valid; accessing any index throws `ArrayIndexOutOfBoundsException`.
- ▶ Maximum size: `Integer.MAX_VALUE` (2147483647), but `OutOfMemoryError` is likely.
- ▶ Every array in Java is an object (created with `new` keyword).
- ▶ `length` is a final variable for arrays (not a method). `length()` is a method for strings.

Default Values on Creation

When an array is created, elements are automatically initialized:

Element Type	Default Value
int / byte / short / long	0
float / double	0.0
char	\u0000
boolean	false
Object reference	null

Multi-Dimensional Arrays

- ▶ Java implements multi-dimensional arrays as arrays of arrays (jagged arrays allowed).
- ▶ Each row can have a different length.
- ▶ `arr.length` gives number of rows (base dimension) only.
- ▶ `arr[0].length` gives column count for first row.

```
// Jagged array
int[][] arr = { {1,2,3}, {4,5}, {6,7,8,9} };
System.out.println(arr.length);    // 3
System.out.println(arr[0].length); // 3
```

Anonymous Arrays

- No reference variable — used for one-time use (e.g., method arguments).
- Cannot specify size when creating anonymous array.

```
sum(new int[]{10, 20, 30}); // Anonymous array
```

Array Covariance & Polymorphism

- Object type arrays: can store subclass objects (follows polymorphism).
- Storing wrong type at runtime → `ArrayStoreException`.
- `Child[]` can be assigned to `Parent[]` (arrays are covariant).
- Element promotion: `char` → `int` (individual), but `char[]` cannot be assigned to `int[]`.

Array Assignment Rules

- Assigning one array to another only copies the reference (not elements).
- Both variables point to the same array object in memory.
- To copy: use `clone()`, `Arrays.copyOf()`, or `System.arraycopy()`.
- Dimension must match: `int[]` and `int[][]` cannot be assigned to each other.

Interview Cheatsheet — Arrays

Method	Class	Description
<code>Arrays.sort(arr)</code>	<code>java.util.Arrays</code>	Sort array in ascending order ($O(n \log n)$)
<code>Arrays.sort(arr, comparator)</code>	<code>java.util.Arrays</code>	Sort with custom comparator
<code>Arrays.binarySearch(arr, key)</code>	<code>java.util.Arrays</code>	Binary search on sorted array; returns index or negative
<code>Arrays.fill(arr, value)</code>	<code>java.util.Arrays</code>	Fill entire array with a value
<code>Arrays.copyOf(arr, len)</code>	<code>java.util.Arrays</code>	Copy array to new length
<code>Arrays.copyOfRange(arr, from, to)</code>	<code>java.util.Arrays</code>	Copy slice of array
<code>Arrays.equals(arr1, arr2)</code>	<code>java.util.Arrays</code>	Compare two arrays element-by-element
<code>Arrays.asList(arr)</code>	<code>java.util.Arrays</code>	Convert array to fixed-size List
<code>Arrays.toString(arr)</code>	<code>java.util.Arrays</code>	String representation of 1D array
<code>Arrays.deepToString(arr)</code>	<code>java.util.Arrays</code>	String representation of multi-dim array
<code>Arrays.stream(arr)</code>	<code>java.util.Arrays</code>	Convert to Stream (Java 8+)
<code>System.arraycopy(src,i,dst,j,n)</code>	<code>System</code>	Efficient native array copy
<code>arr.length</code>	-	Size of array (field, not method)
<code>arr.clone()</code>	-	Shallow copy of array

3. Variables & Var-Args

Variable Categories

Based on position, Java has three types of variables:

Type	Declared In	Storage	Default	Thread-Safe?
Instance Variable	Class body (non-static)	Heap	Yes (JVM provides)	No
Static Variable	Class body (static)	Method Area	Yes (JVM provides)	No
Local Variable	Method / Constructor / Block	Stack	No — must initialize	Yes (each thread has own copy)

Access Rules

- ▶ Instance members cannot be accessed directly from static context.
- ▶ Static members can be accessed from both static and instance areas.
- ▶ Local variables have method-level scope; no access modifiers allowed (except final).
- ▶ If a local variable and an instance variable share a name, use `this.var` for instance and `ClassName.var` for static.

Static vs Instance Variables

- ▶ Static variables: one copy shared by all objects. Accessed via class name (recommended).
- ▶ Instance variables: each object has its own copy. Accessed via object reference.
- ▶ For static final variables: initialized in static block or at declaration — never in constructor.
- ▶ For instance final variables: must be initialized in declaration, constructor, or instance block.

Variable Declaration with Data Types

Combination	Example	Note
Instance + Primitive	<code>int age;</code>	Stored in heap; default 0
Instance + Reference	<code>String name;</code>	Default null
Static + Primitive	<code>static int count;</code>	Shared across instances
Static + Reference	<code>static List<T> list;</code>	Shared across instances
Local + Primitive	<code>int x = 5;</code>	Must initialize before use
Local + Reference	<code>String s = "hi";</code>	Must initialize before use

Var-Args (Variable Arguments — Java 5+)

```
void displayNumbers(int... numbers) {  
    for (int num : numbers) { ... }  
}  
displayNumbers();           // 0 arguments  
displayNumbers(10,20,30); // multiple arguments
```

- ▶ Internally treated as an array. Accessible via `.length`.
- ▶ Only ONE var-arg parameter per method; must be the LAST parameter.
- ▶ Var-arg methods have LEAST priority in method resolution (after exact, widening, boxing).
- ▶ Cannot declare both `void m1(int... a)` and `void m1(int[] a)` — compile-time duplicate error.
- ▶ Can replace `String[]` args with `String...` args in main method.



Interview Cheatsheet — Variables & Memory

Concept / Keyword	Usage
final variable	Value assigned once; cannot be changed
static final	Class-level constant; initialized at declaration or static block
volatile	Always read/write from main memory; no caching in threads
transient	Excluded from serialization
this.varName	Refers to instance variable when shadowed by local
ClassName.varName	Access static variable
int... args	Var-arg parameter (must be last, only one per method)
Thread.currentThread()	Get current executing thread
ThreadLocal<T>	Per-thread variable (no synchronization needed)

4. Operators & Assignments

Operator Precedence (Highest → Lowest)

Level	Operators
Postfix	expr++, expr--
Unary	++expr, --expr, +, -, ~, !
Cast	(type)
Multiplicative	*, /, %
Additive	+, -
Shift	<<, >>, >>>
Relational	<, <=, >, >=, instanceof
Equality	==, !=
Bitwise AND	&
Bitwise XOR	^
Bitwise OR	
Logical AND	&&
Logical OR	
Ternary	?:
Assignment	=, +=, -=, *=, /=, %=, &=, ^=, =, <<=, >>=, >>>=

Same-precedence operators: left-to-right (except assignment & ternary = right-to-left).

Increment / Decrement

- ▶ ++x: pre-increment (increment first, then use)
- ▶ x++: post-increment (use first, then increment)
- ▶ Cannot apply to final variables or constants (5++ → CE).
- ▶ Applicable to: byte, short, int, long, float, double, char.
- ▶ Internally performs typecasting: b++ → b = (byte)(b + 1).

Type Promotion in Arithmetic

Result type = max(int, Type1, Type2):

Operation	Result
byte + byte	int
short + short	int

Operation	Result
char + char	int
int + float	float
long + float	float
float + double	double

Division & Modulo Behavior

- ▶ Integral division by zero (10/0) → ArithmeticException at runtime.
- ▶ Floating-point division by zero: 10/0.0 → Infinity; -10/0.0 → -Infinity.
- ▶ 0/0.0 → NaN (Not a Number).
- ▶ Any comparison with NaN always returns false; != NaN always returns true.
- ▶ Only / and % operators cause ArithmeticException.

Bitwise Operators

Operator	Symbol	Description
Bitwise AND	&	Result is 1 if both bits are 1; works on int and boolean
Bitwise OR		Result is 1 if at least one bit is 1
Bitwise XOR	^	Result is 1 if bits are different
Bitwise NOT	~	Inverts all bits; only for integral types (not boolean)
Logical NOT	!	Only for boolean types
Signed Right Shift	>>	Fills with sign bit (preserves negative)
Unsigned Right Shift	>>>	Always fills with 0 (ignores sign)
Left Shift	<<	Shifts bits left; fills right with 0

Equality (== vs equals())

Feature	== Operator	equals () Method
Compares	References (memory addresses)	Content (if overridden)
Works for	Primitives and objects	Objects only
Default behavior	Identity check	Identity check (Object); overridable
With null	obj == null → true/false	Can throw NullPointerException if obj is null
Unrelated types	Compile-time error	Returns false

instanceof Operator

- Checks whether an object is an instance of a class or interface.
- Requires parent-child relationship; otherwise CE.
- null instanceof AnyClass always returns false.

```
if (obj instanceof String) { String s = (String) obj; }
```

Type Casting

- Implicit (widening): byte → short → int → long → float → double; char → int.
- Explicit (narrowing): double → float → long → int → short → byte; may lose data.
- Compound assignment operators (+=, -=, etc.) internally perform implicit typecasting.

Interview Cheatsheet — Operators & Casting

Concept	Quick Summary
<code>x >>> 2</code>	Unsigned right shift — always fills with 0 (useful for hash computations)
<code>a ^ b</code>	XOR: swap without temp: <code>a^=b; b^=a; a^=b;</code>
<code>n & 1</code>	Check if odd (result = 1) or even (result = 0)
<code>n >> 1</code>	Fast divide by 2
<code>n << 1</code>	Fast multiply by 2
<code>Double.isNaN(x)</code>	Check for NaN (never use <code>==</code> for NaN)
<code>Double.isInfinite(x)</code>	Check for positive/negative infinity
<code>Math.abs(n)</code>	Absolute value
<code>Math.max(a,b) / Math.min(a,b)</code>	Max / Min of two values
<code>Math.pow(a,b)</code>	a raised to power b (returns double)
<code>Math.sqrt(n)</code>	Square root
<code>Integer.bitCount(n)</code>	Count set bits (1s) in binary representation
<code>Integer.highestOneBit(n)</code>	Highest power of 2 ≤ n
<code>Integer.numberOfLeadingZeros(n)</code>	Leading zero count (useful for log2)
<code>Integer.reverse(n)</code>	Reverse bits of integer

5. Flow Control

Selection Statements

if / if-else

- ▶ Condition must evaluate to boolean. No auto-conversion (unlike C/C++).
- ▶ else block is optional. Curly braces optional for single-statement body.
- ▶ Declaration statement not allowed directly after if without braces → CE.
- ▶ Java does NOT have "dangling else" problem: else always binds to nearest preceding if.

switch Statement

- ▶ Supported types: byte, short, char, int, Byte, Short, Character, Integer, String, enum.
- ▶ NOT supported: long, float, double (and their wrappers).
- ▶ Outer {} are MANDATORY for switch.
- ▶ case and default are optional. Duplicate case labels → CE.
- ▶ case label must be a compile-time constant expression; variables (non-final) → CE.
- ▶ Fall-through: without break, execution continues to next case.
- ▶ default can appear anywhere but conventionally placed last.
- ▶ Java 12+: Enhanced switch with case -> (arrow) syntax prevents fall-through.

Iterative Statements

Loop	Type	When to Use
for	Entry-controlled	Known number of iterations
while	Entry-controlled	Unknown iterations; condition checked first
do-while	Exit-controlled	Execute at least once; condition checked after
for-each	Enhanced for	Iterate arrays/Iterable collections (Java 5+)

for Loop Details

- ▶ All three parts (initialization, condition, update) are optional: for(;;) = infinite loop.
- ▶ Multiple initializations allowed (same type only): for(int i=0, j=5; i<5; i++, j--).
- ▶ Initialization can contain any valid Java statement (including method calls).
- ▶ Condition must be boolean; defaults to true if omitted.

for-each Loop

- ▶ Works with arrays and any class implementing Iterable.
- ▶ Cannot modify collection size during iteration → ConcurrentModificationException.
- ▶ No index access — use traditional for loop if index is needed.

- Arrays do NOT implement Iterable; compiler handles them specially.

Transfer Statements

break

- Terminates current loop or switch block.
- Labeled break: exits any labeled block (not just loops).
- break outside loop/switch → CE.

continue

- Skips current iteration and jumps to next.
- In do-while: continue jumps to the while condition check.
- Labeled continue: skips iteration of specified outer loop.
- continue in if/else or switch (outside loop) → CE.

Unreachable Statements

- Identifying unreachable code is the compiler's job (not JVM).
- while(true){} followed by any statement → CE: unreachable statement.
- Comparing two final compile-time constants in while condition → CE.

Interview Cheatsheet — Flow Control & Loops

Pattern	Code Snippet	Use Case
Infinite loop	<code>while(true) { if(cond) break; }</code>	Processing until condition met
For + two vars	<code>for(int i=0,j=n-1; i<j; i++,j--)</code>	Two-pointer technique
Labeled break	<code>outer: for(...) { for(...) { break outer; } }</code>	Exit nested loops
Switch expression (Java 14)	<code>int r = switch(x) { case 1 -> 10; default -> 0; };</code>	Cleaner multi-branch
for-each array	<code>for(int n : arr) { sum += n; }</code>	Simple array traversal
Iterator safe remove	<code>while(it.hasNext()) { if(cond) it.remove(); }</code>	Avoid CME

6. Object-Oriented Programming

Four Pillars of OOP

Pillar	Definition	Achieved Via
Data Hiding	Restrict direct access to data from outside	private fields + public getters/setters
Abstraction	Hide implementation; show only essentials	Abstract classes, Interfaces
Encapsulation	Bundle data + behavior; control access	Access modifiers (private, protected, public)
Polymorphism	Object/method taking multiple forms	Overloading (compile-time), Overriding (runtime)

IS-A (Inheritance) vs HAS-A (Composition)

Relationship	Keyword	Description	Prefer When
IS-A	extends / implements	Subclass inherits parent; "Dog IS-A Animal"	Need ALL parent functionality
HAS-A (Composition)	new / injection	Strong association; cannot exist independently	Need SPECIFIC functionality
HAS-A (Aggregation)	Reference passed in	Weak association; can exist independently	Loose coupling needed

Method Overloading

- ▶ Same method name, different parameter list (number, type, or order).
- ▶ Return type alone cannot differentiate overloaded methods.
- ▶ Resolved at compile-time (compile-time / static polymorphism / early binding).
- ▶ Resolution order: Exact match → Widening → Boxing/Unboxing → Var-args (least priority).
- ▶ Child type argument preferred over parent type (most specific method wins).
- ▶ Can occur in same class or between parent and child class.

Method Overriding

- ▶ Same name, return type (or covariant), and parameter list as parent method.
- ▶ Access modifier cannot be MORE restrictive than parent.
- ▶ Cannot override: static, final, or private methods.
- ▶ Resolved at runtime (runtime polymorphism / dynamic binding / late binding).
- ▶ @Override annotation: recommended for compile-time error detection.
- ▶ Covariant return type (Java 5+): child can return subtype of parent's return type (objects only, not primitives).
- ▶ Exception: overriding method cannot throw broader checked exceptions.

Overriding vs Method Hiding

Aspect	Overriding	Method Hiding
Applicable to	Instance methods	Static methods
Polymorphism	Runtime polymorphism	No runtime polymorphism
Binding	Dynamic (runtime)	Static (compile-time)
Resolution	Based on actual object type	Based on reference type
@Override	Used	Not typically used

Variable Resolution

- ▶ Variables CANNOT be overridden — only hidden.
- ▶ Variable resolution is ALWAYS based on the reference type at compile time.
- ▶ Instance variables: resolved using reference type (not runtime object type).

Inheritance Rules

- ▶ Java supports: Single, Multilevel, Hierarchical inheritance.
- ▶ Multiple inheritance via classes NOT supported (diamond problem).
- ▶ Multiple inheritance achieved via interfaces.
- ▶ Cyclic inheritance: not allowed in Java.
- ▶ Object class is the root of all Java classes (every class directly/indirectly extends it).
- ▶ When no class is extended, Java auto-extends Object.

Constructors

- ▶ Used to initialize objects; same name as class, no return type.
- ▶ Compiler generates default constructor only if NO constructor is explicitly defined.
- ▶ First statement must be this() or super() (compiler auto-inserts super() if omitted).
- ▶ Cannot use both this() and super() simultaneously in same constructor.
- ▶ this() and super() allowed ONLY in constructors (not in methods or blocks).
- ▶ Constructors are NOT inherited; hence not overridable.
- ▶ Applicable modifiers: public, protected, private, default.
- ▶ If parent has only parameterized constructor, child MUST call super(args).

Static & Instance Flow Control

Static Flow Control (class loading)

- ▶ 1. Identify static members → 2. Execute static declarations/blocks in order → 3. Execute main().
- ▶ Static blocks executed once when class is loaded.
- ▶ Write-only variable: direct read before declaration → "illegal forward reference" CE.

- ▶ Indirect read via static method: allowed (returns default value).
- ▶ Parent class static flow executes before child class static flow.

Instance Flow Control (object creation)

- ▶ 1. Identify instance members → 2. Execute instance blocks/declarations → 3. Execute constructor.
- ▶ Occurs every time an object is created.
- ▶ Parent constructor executes before child constructor (super() is first statement).

Object Creation

Method	Description
new Keyword	Most common; <code>ClassName obj = new ClassName()</code>
<code>Class.forName().newInstance()</code>	Reflection; deprecated in Java 9+
<code>getDeclaredConstructor().newInstance()</code>	Modern reflection approach
Factory Method	<code>ClassName.getInstance()</code> pattern (Singleton)
<code>clone()</code>	Cloning existing object (requires Cloneable)
Deserialization	Restoring from byte stream (ObjectInputStream)

Coupling & Cohesion

- ▶ Coupling: degree of dependency between components. LOOSE coupling is preferred.
- ▶ Cohesion: degree of focused functionality in a component. HIGH cohesion is preferred.
- ▶ Tight coupling → hard to maintain, test, and modify.
- ▶ High cohesion → better maintainability, reusability, readability.

Abstract Classes

- ▶ Cannot be instantiated; meant to be subclassed.
- ▶ Can contain both abstract methods (no body) and concrete methods.
- ▶ If a class has even ONE abstract method, the class must be abstract.
- ▶ abstract + final is illegal (contradictory purposes).
- ▶ abstract method cannot be: final, static, private, native, synchronized, strictfp.
- ▶ Abstract class can have constructors (called when subclass is instantiated).

Singleton Pattern

```
// Eager initialization
public class Singleton {
    private static final Singleton INSTANCE = new Singleton();
    private Singleton() {}
    public static Singleton getInstance() { return INSTANCE; }
}
```



```
// Lazy initialization with double-checked locking
public class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Interview Cheatsheet — OOP & Object Methods

Method / Concept	Class	Description
toString()	Object	Override for meaningful string; default: ClassName@hexHashCode
equals(Object)	Object	Override for content equality; default: reference equality
hashCode()	Object	Override alongside equals(); same content → same hash
getClass()	Object	Returns runtime Class object
clone()	Object	Shallow copy; requires Cloneable
finalize()	Object	Called by GC before destruction (deprecated in Java 9)
instanceof	-	Type check; null instanceof AnyClass = false
this()	Constructor	Call another constructor in same class (must be first)
super()	Constructor	Call parent constructor (must be first)
@Override	Annotation	Compile-time check for correct overriding
Objects.equals(a,b)	Objects	Null-safe equality check (Java 7+)
Objects.hash(fields...)	Objects	Generate hashCode from multiple fields
Objects.requireNonNull(obj)	Objects	Throw NPE with message if null

7. Interfaces & Modifiers

Interface Fundamentals

- ▶ An interface defines a contract between client and service provider.
- ▶ All methods: implicitly public and abstract (before Java 8).
- ▶ All variables: implicitly public, static, and final.
- ▶ Cannot be instantiated; cannot have constructors.
- ▶ A class can implement multiple interfaces (multiple inheritance).
- ▶ An interface can extend multiple interfaces.
- ▶ extends comes before implements in class declaration.

Java 8+ Interface Enhancements

Feature	Java Version	Description
default methods	Java 8	Methods with body; implementing class can override
static methods	Java 8	Belong to interface; called via <code>InterfaceName.method()</code>
private methods	Java 9	Helper methods for default methods (not inherited)
Functional Interface	Java 8	Exactly one abstract method; enables lambda expressions

Access Modifiers Summary

Modifier	Same Class	Same Package	Subclass (other pkg)	Everywhere
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓ (via inheritance)	✗
public	✓	✓	✓	✓

Non-Access Modifiers

Modifier	Applicable To	Effect
static	Methods, variables, nested classes, blocks	Belongs to class; no instance needed
final	Variables, methods, classes	Immutable value / prevent override / prevent inheritance
abstract	Classes, methods	Must be implemented; class cannot be instantiated
synchronized	Methods, blocks	Thread-safe; one thread at a time

Modifier	Applicable To	Effect
volatile	Variables	Always read from main memory; no thread caching
transient	Variables	Excluded from serialization
native	Methods	Implemented in native code (C/C++ via JNI)
strictfp	Classes, interfaces, methods	Ensures IEEE 754 floating-point compliance

Where Modifiers Are Applicable

Target	Allowed Modifiers
Outer class	public, default (package-private), abstract, final, strictfp
Inner class	public, protected, private, default, static, abstract, final, strictfp
Outer interface	public, default, abstract, strictfp
Methods	All except volatile and transient
Variables	All except abstract, synchronized, native, strictfp
Constructors	public, protected, private, default
Blocks	static, synchronized only

Illegal Modifier Combinations

Combination	Reason Illegal
abstract + final	abstract requires override; final prevents override
abstract + private	abstract requires subclass access; private denies it
abstract + static	abstract must be overridden; static has implementation
abstract + synchronized	abstract has no body; synchronized requires method body
abstract + native	abstract has no body; native implies native implementation
abstract + strictfp	abstract has no body; strictfp applies to implementation
volatile + final	volatile implies change; final prevents change
native + strictfp	native code has no IEEE 754 guarantee

Marker Interfaces

- Contains no methods or fields — acts as a tag.
- JVM or frameworks check presence at runtime to enable special behavior.

Marker Interface	Purpose
Serializable	Enables object serialization
Cloneable	Allows <code>object.clone()</code> without <code>CloneNotSupportedException</code>

Marker Interface	Purpose
Remote (RMI)	Marks objects accessible via Remote Method Invocation
RandomAccess	Signals fast random access (e.g., ArrayList)

Packages

- ▶ Package declaration must be first non-comment line in the file.
- ▶ Order: Package → Imports → Class declarations.
- ▶ Compilation with packages: `javac -d . FileName.java` (-d creates directory structure).
- ▶ Execution: `java com.example.ClassName` (use fully qualified name).
- ▶ `java.lang` is automatically imported.
- ▶ `import java.util.*` imports all classes in `java.util` but NOT sub-packages.
- ▶ Ambiguity with same-name classes from different packages → CE; use fully qualified name.

Interview Cheatsheet — Interfaces & Modifiers

Concept	Quick Rule / Code
Functional Interface	@FunctionalInterface — exactly 1 abstract method
Lambda shorthand	<code>Interf i = () -> System.out.println("hi");</code>
Default method	<code>default void greet() { System.out.println("hello"); }</code>
Static method call	<code>MyInterface.staticMethod();</code> (not via implementor)
Interface variable	Implicitly public static final — must initialize
final class	Cannot be extended (e.g., String, Integer)
final method	Cannot be overridden
Abstract class constructor	Exists; called via <code>super()</code> from subclass
private constructor	Prevents subclassing and external instantiation
Adapter class	Abstract class implementing interface with empty method bodies

8. Strings, StringBuffer & StringBuilder

String Fundamentals

- ▶ String is immutable — any modification creates a new String object.
- ▶ String Constant Pool (SCP): special heap area for string literals; avoids duplicate objects.
- ▶ String s = "Java" → 1 object in SCP. String s = new String("Java") → 1 in heap + possibly 1 in SCP.
- ▶ SCP objects are not garbage collected until JVM shuts down.
- ▶ "Hello" + " World" with compile-time constants → compiler concatenates at compile time.
- ▶ Runtime string concatenation → new object in heap (not SCP).
- ▶ intern() method: moves string to SCP and returns SCP reference.

String vs StringBuffer vs StringBuilder

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes (immutable)	Yes (synchronized)	No
Performance	Slowest for modification	Moderate	Fastest
Default capacity	N/A	16 chars	16 chars
Growth formula	N/A	(capacity × 2) + 2	(capacity × 2) + 2
Package	java.lang	java.lang	java.lang
equals() overridden	Yes (content)	No (reference)	No (reference)
toString() overridden	Yes	Yes	Yes
Introduced	Java 1.0	Java 1.0	Java 1.5

String Class Key Methods

Method	Return Type	Description
length()	int	Number of characters
charAt(int index)	char	Character at index
indexOf(char/String)	int	First occurrence index; -1 if not found
lastIndexOf(char/String)	int	Last occurrence index; -1 if not found
substring(begin)	String	Substring from begin to end
substring(begin, end)	String	Substring [begin, end)
toUpperCase()	String	All uppercase

Method	Return Type	Description
<code>toLowerCase()</code>	String	All lowercase
<code>trim()</code>	String	Remove leading/trailing whitespace
<code>strip()</code> [Java 11]	String	Unicode-aware whitespace removal
<code>replace(old, new)</code>	String	Replace all occurrences of char or CharSequence
<code>replaceAll(regex, str)</code>	String	Replace using regex
<code>contains(CharSequence)</code>	boolean	Check if string contains sequence
<code>startsWith(prefix)</code>	boolean	Check prefix
<code>endsWith(suffix)</code>	boolean	Check suffix
<code>equals(Object)</code>	boolean	Content equality (case-sensitive)
<code>equalsIgnoreCase(String)</code>	boolean	Content equality (case-insensitive)
<code>compareTo(String)</code>	int	Lexicographic comparison; <0, 0, or >0
<code>compareToIgnoreCase(String)</code>	int	Lexicographic comparison ignoring case
<code>split(regex)</code>	String[]	Split by pattern
<code>split(regex, limit)</code>	String[]	Split with limit on result count
<code>toCharArray()</code>	char[]	Convert to character array
<code>valueOf(primitive)</code>	String	Convert primitive to String
<code>format(format, args)</code>	String	String formatting (printf-style)
<code>isEmpty()</code>	boolean	True if <code>length() == 0</code>
<code>isBlank()</code> [Java 11]	boolean	True if empty or only whitespace
<code>join(delim, parts)</code>	String	Join strings with delimiter
<code>repeat(n)</code> [Java 11]	String	Repeat string n times
<code>intern()</code>	String	Return SCP reference
<code>matches(regex)</code>	boolean	Check if whole string matches regex
<code>chars()</code> [Java 8]	IntStream	Stream of char values

StringBuffer / StringBuilder Methods

Method	Description
<code>append(data)</code>	Add to end (overloaded for all types)
<code>insert(index, data)</code>	Insert at position
<code>delete(start, end)</code>	Remove chars [start, end)
<code>deleteCharAt(index)</code>	Remove char at index
<code>replace(start, end, str)</code>	Replace [start, end) with str
<code>reverse()</code>	Reverse the character sequence
<code>charAt(index)</code>	Get character at index

Method	Description
<code>setCharAt(index, char)</code>	Set character at index
<code>indexOf(str)</code>	First occurrence of substring
<code>substring(start)</code>	Substring from start
<code>substring(start, end)</code>	Substring [start, end)
<code>length()</code>	Number of characters
<code>capacity()</code>	Current buffer capacity
<code>ensureCapacity(min)</code>	Ensure at least min capacity
<code>setLength(len)</code>	Truncate or pad to length
<code>trimToSize()</code>	Reduce capacity to current length
<code>toString()</code>	Convert to String

Interview Cheatsheet — String Techniques

Pattern	Code	Use Case
Reverse a string	<code>new StringBuilder(s).reverse().toString()</code>	Palindrome check
Count occurrences	<code>s.split(ch, -1).length - 1</code>	Character frequency
Check palindrome	<code>s.equals(new StringBuilder(s).reverse().toString())</code>	Palindrome
Remove spaces	<code>s.replace(" ", "")</code>	Cleaning input
Split by whitespace	<code>s.trim().split("\\s+")</code>	Tokenize sentence
Convert to char[]	<code>s.toCharArray()</code>	Frequency counting, anagram
Char frequency map	<code>for(char c: s.toCharArray()) map.merge(c, 1, Integer::sum)</code>	Anagram / frequency
First non-repeat char	<code>for(char c: s.toCharArray()) if(map.get(c)==1) return c;</code>	Classic interview Q
String to int	<code>Integer.parseInt(s)</code>	Safe parsing
int to String	<code>String.valueOf(n)</code> or <code>Integer.toString(n)</code>	Digit manipulation
Longest common prefix	<code>while(!s2.startsWith(prefix)) prefix=prefix.substring(0,len-1)</code>	LCP
String comparison	<code>s1.compareTo(s2)</code> -- never use <code>==</code>	Always use <code>equals()</code>

9. Collections Framework

Why Collections Over Arrays?

Feature	Arrays	Collections
Size	Fixed	Dynamic
Element types	Homogeneous only	Heterogeneous (without generics)
Built-in methods	None	Rich API (sort, search, etc.)
Primitives	Supported directly	Wrapper classes needed
Memory	Efficient (fixed)	Some overhead for resizing

Core Collection Interfaces

Interface	Description	Key Implementations
Collection	Root of collection hierarchy	No direct implementation
List	Ordered, allows duplicates	ArrayList, LinkedList, Vector
Set	No duplicates	HashSet, LinkedHashSet, TreeSet
SortedSet	Sorted Set	TreeSet
NavigableSet	SortedSet + navigation methods	TreeSet
Queue	FIFO processing	LinkedList, PriorityQueue, ArrayDeque
Deque	Double-ended queue	ArrayDeque, LinkedList
Map	Key-value pairs (not Collection)	HashMap, LinkedHashMap, TreeMap
SortedMap	Map sorted by keys	TreeMap
NavigableMap	SortedMap + navigation	TreeMap

List Implementations

Feature	ArrayList	LinkedList	Vector
Underlying DS	Resizable array	Doubly linked list	Resizable array
Random access	$O(1)$ — fast	$O(n)$ — slow	$O(1)$ — fast
Insert/delete (middle)	$O(n)$ — slow (shifting)	$O(1)$ — fast (node update)	$O(n)$ — slow
Thread-safe	No	No	Yes (all methods synchronized)
Growth	1.5x current capacity	Node-by-node	Doubles capacity

Feature	ArrayList	LinkedList	Vector
Default capacity	10	N/A	10
RandomAccess	Yes	No	Yes
Best for	Frequent reads	Frequent inserts/deletes	Multi-threaded legacy

Set Implementations

Feature	HashSet	LinkedHashSet	TreeSet
Underlying DS	HashMap (hash table)	HashMap + linked list	Red-Black tree
Order	No order	Insertion order	Natural/comparator order
Null allowed	1 null	1 null	No null (throws NPE)
Duplicates	No	No	No
Heterogeneous	Yes	Yes	No (CCE at runtime)
Performance	O(1) add/remove	O(1) add/remove	O(log n) all ops
Introduced	Java 1.2	Java 1.4	Java 1.2

Map Implementations

Feature	HashMap	LinkedHashMap	TreeMap	Hashtable
Order	No order	Insertion order	Sorted by key	No order
Null key	1 null key	1 null key	No null key	No null
Thread-safe	No	No	No	Yes
Performance	O(1)	O(1)	O(log n)	O(1) slower
Default capacity	16, 0.75 LF	16, 0.75 LF	N/A	11, 0.75 LF
Introduced	Java 1.2	Java 1.4	Java 1.2	Java 1.0 (legacy)

Queue & Deque

Method	Exception if fail	Returns null/false if fail
add(e)	Yes	offer(e)
remove()	Yes	poll()
element()	Yes	peek()
Deque: push(e)	Yes	offerFirst(e)
Deque: pop()	Yes	pollFirst()
Deque: peekFirst()	No	returns null

PriorityQueue

- Elements ordered by natural ordering or custom Comparator.
- Default: min-heap (smallest element at head).
- No null elements allowed. Duplicates allowed.
- Time complexity: $O(\log n)$ insert/remove, $O(1)$ peek.

```
// Max-heap using PriorityQueue
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
```

Comparable vs Comparator

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o)	compare(T o1, T o2)
Return	int (<0, 0, >0)	int (<0, 0, >0)
Modifying class	Required (implement in class)	Not required (separate class/lambda)
Natural order?	Yes (one default order)	Multiple sort orders possible
Usage	TreeSet, TreeMap default sort	Pass to constructor or sort methods

Concurrent Collections

Class	Thread-safe?	Description
ConcurrentHashMap	Yes	Segment/CAS locking; no null keys/values
CopyOnWriteArrayList	Yes	Write creates new copy; fail-safe iterator
CopyOnWriteArraySet	Yes	Backed by CopyOnWriteArrayList
ConcurrentSkipListMap	Yes	Sorted concurrent map (TreeMap equivalent)
ConcurrentSkipListSet	Yes	Sorted concurrent set
ArrayBlockingQueue	Yes	Bounded blocking queue (FIFO)
LinkedBlockingQueue	Yes	Optionally bounded blocking queue

Cursors (Traversal Mechanisms)

Cursor	Direction	Remove?	Add/Set?	Works With
Enumeration	Forward only	No	No	Legacy (Vector, Hashtable)
Iterator	Forward only	Yes	No	All Collection types

Cursor	Direction	Remove?	Add/Set?	Works With
ListIterator	Forward & backward	Yes	Yes (add, set)	List types only

Collections Utility Class

Method	Description
<code>Collections.sort(list)</code>	Sort in natural order
<code>Collections.sort(list, comparator)</code>	Sort with custom comparator
<code>Collections.binarySearch(list, key)</code>	Binary search on sorted list
<code>Collections.reverse(list)</code>	Reverse list in place
<code>Collections.shuffle(list)</code>	Random shuffle
<code>Collections.min(coll) / max(coll)</code>	Find minimum / maximum
<code>Collections.frequency(coll, obj)</code>	Count occurrences of element
<code>Collections.synchronizedList(list)</code>	Thread-safe list wrapper
<code>Collections.unmodifiableList(list)</code>	Read-only list
<code>Collections.emptyList()</code>	Immutable empty list
<code>Collections.singletonList(obj)</code>	Immutable single-element list
<code>Collections.nCopies(n, obj)</code>	Immutable list with n copies
<code>Collections.disjoint(c1, c2)</code>	True if no common elements
<code>Collections.addAll(coll, elems)</code>	Add multiple elements at once



Interview Cheatsheet — Collections

Task	Best Collection	Key Method(s)
Ordered list, fast random access	ArrayList	<code>get(i)</code> , <code>set(i,v)</code> , <code>add()</code> , <code>remove()</code>
Frequent insert/delete at ends	LinkedList / ArrayDeque	<code>addFirst()</code> , <code>addLast()</code> , <code>pollFirst()</code>
Unique elements, no order	HashSet	<code>add()</code> , <code>contains()</code> , <code>remove()</code>
Unique, insertion order	LinkedHashSet	<code>add()</code> , <code>contains()</code> , <code>remove()</code>
Unique, sorted	TreeSet	<code>first()</code> , <code>last()</code> , <code>floor()</code> , <code>ceiling()</code>
Key-value, fast lookup	HashMap	<code>put()</code> , <code>get()</code> , <code>getOrDefault()</code> , <code>containsKey()</code>
Key-value, insertion order	LinkedHashMap	<code>put()</code> , <code>get()</code> , <code>entrySet()</code>
Key-value, sorted keys	TreeMap	<code>firstKey()</code> , <code>lastKey()</code> , <code>subMap()</code>
Priority processing	PriorityQueue	<code>offer()</code> , <code>poll()</code> , <code>peek()</code>
Stack operations	ArrayDeque	<code>push()</code> , <code>pop()</code> , <code>peek()</code>
Thread-safe map	ConcurrentHashMap	<code>putIfAbsent()</code> , <code>computeIfAbsent()</code>

Task	Best Collection	Key Method(s)
Count char/word frequency	HashMap	<code>map.merge(key, 1, Integer::sum)</code>
Group elements by key	<code>HashMap<K, List<V>></code>	<code>map.computeIfAbsent(k, x -> new ArrayList<>()).add(v)</code>
Sort map by value	LinkedHashMap + stream	<code>sorted by Map.Entry.comparingByValue()</code>

10. Exception Handling

Exception Hierarchy

```
Throwable
├── Exception (recoverable)
│   ├── IOException (checked)
│   ├── SQLException (checked)
│   └── RuntimeException (unchecked)
│       ├── NullPointerException
│       ├── ArithmeticException
│       ├── ArrayIndexOutOfBoundsException
│       ├── ClassCastException
│       └── NumberFormatException
└── Error (usually unrecoverable)
    ├── OutOfMemoryError
    ├── StackOverflowError
    └── ExceptionInInitializerError
```

Checked vs Unchecked Exceptions

Feature	Checked	Unchecked
Checked at	Compile time	Runtime
Handling	Must handle or declare (throws)	Optional
Recoverable?	Usually (e.g., file not found)	Often programming errors
Superclass	Exception (excl. RuntimeException)	RuntimeException or Error
Examples	IOException, SQLException	NullPointerException, ArithmeticException

Five Keywords

Keyword	Purpose
try	Contains risky code that might throw exception
catch	Handles specific exception type(s)
finally	Always executes (cleanup); even with return; only System.exit() prevents it
throw	Explicitly throw an exception object
throws	Declares exceptions a method might throw (delegates to caller)

try-catch-finally Rules

- ▶ try must be followed by catch or finally (or both).
- ▶ Curly braces {} are mandatory for all blocks.
- ▶ Multiple catch blocks: more specific before general (else CE).

- ▶ At most ONE finally block; finally must come after all catch blocks.
- ▶ If exception occurs in try, remaining statements in try are skipped.
- ▶ Multi-catch (Java 7): `catch(IOException | SQLException e)` — exceptions must not be parent-child.
- ▶ Exception variable in multi-catch is implicitly final.
- ▶ finally return overrides try/catch return.

throw vs throws

Aspect	throw	throws
Used to	Actually throw an exception object	Declare potential exceptions
Follows	An exception object	Exception class name(s)
Code after throw	Unreachable → CE	N/A
Type required	Throwable subclass	Throwable subclass
Checked exception	Must catch or declare	Informs caller to handle

Try-With-Resources (Java 7+)

```
try (FileReader fr = new FileReader("file.txt");
    BufferedReader br = new BufferedReader(fr)) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
// Resource auto-closed; implements AutoCloseable
```

- ▶ Resources are implicitly final — cannot be reassigned inside try.
- ▶ Multiple resources: separated by semicolons.
- ▶ AutoCloseable interface (Java 7): contains only `close()` method.
- ▶ Java 9: can use effectively final variables declared outside try.

Custom Exceptions

```
// Checked custom exception
class InsufficientFundsException extends Exception {
    InsufficientFundsException(String message) { super(message); }
}

// Unchecked custom exception (recommended)
class InvalidAgeException extends RuntimeException {
    InvalidAgeException(String message) { super(message); }
}
```

- ▶ Extend `RuntimeException` for unchecked (recommended — less boilerplate).
- ▶ Extend `Exception` for checked (caller must handle or declare).
- ▶ Call `super(message)` to pass description to default exception handler.

JVM vs Programmatic Exceptions

Category	Examples
JVM Exceptions	NullPointerException, ClassCastException, StackOverflowError, ArrayIndexOutOfBoundsException, OutOfMemoryError
Programmatic	IllegalArgumentException, NumberFormatException, IllegalStateException, custom exceptions

final vs finally vs finalize

Keyword	Type	Purpose
final	Modifier	Immutable variable / prevent override / prevent inheritance
finally	Block	Cleanup; always runs after try-catch
finalize()	Method	Called by GC before object destruction (deprecated Java 9)



Interview Cheatsheet — Exception Handling

Method / Concept	Class	Description
getMessage()	Throwable	Exception message only
toString()	Throwable	Class name + message
printStackTrace()	Throwable	Full stack trace to stderr
getCause()	Throwable	Original cause (wrapped exception)
initCause(Throwable)	Throwable	Set the cause
getStackTrace()	Throwable	Array of StackTraceElement
AutoCloseable.close()	AutoCloseable	Auto-called by try-with-resources
Objects.requireNonNull(obj)	Objects	Throws NPE with message
Optional.of(val)	Optional	Non-null value; orElse() / orElseThrow()

11. Multithreading

Thread Lifecycle

State	Description
New	Thread created with new; not yet started
Runnable	start() called; waiting for CPU time
Running	Thread executing; CPU allocated
Blocked/Waiting	Waiting for lock, resource, or notification
Timed Waiting	Waiting with timeout (sleep, join with millis)
Terminated	Execution complete or exception occurred

Creating Threads

Method 1: Extend Thread

```
class MyThread extends Thread {  
    @Override public void run() { /* task */ }  
}  
MyThread t = new MyThread();  
t.start(); // Always call start(), NOT run()
```

Method 2: Implement Runnable (preferred)

```
class MyTask implements Runnable {  
    @Override public void run() { /* task */ }  
}  
Thread t = new Thread(new MyTask());  
t.start();  
  
// Lambda (Java 8+)  
Thread t2 = new Thread(() -> { /* task */ });  
t2.start();
```

- ▶ Runnable preferred: class can extend another class; better separation of concerns.
- ▶ start() creates new thread and calls run(); calling run() directly executes in current thread.
- ▶ Calling start() on already-started thread → `IllegalThreadStateException`.

Thread Control Methods

Method	Type	Effect
<code>Thread.yield()</code>	static	Pause current thread; give same/higher priority threads a chance (hint only)
<code>t.join()</code>	instance	Current thread waits for t to finish

Method	Type	Effect
t.join(millis)	instance	Wait at most millis milliseconds for t
Thread.sleep(millis)	static	Pause current thread; does NOT release lock
t.interrupt()	instance	Interrupt sleeping/waiting thread; throws InterruptedException
t.setPriority(n)	instance	Set priority (1-10); MIN=1, NORM=5, MAX=10
t.getPriority()	instance	Get current priority
Thread.currentThread()	static	Reference to currently executing thread
t.getName() / setName()	instance	Get/set thread name
t.isDaemon()	instance	True if daemon thread
t.setDaemon(true)	instance	Set daemon BEFORE start()

Synchronization

- ▶ Prevents data inconsistency when multiple threads access shared resources.
- ▶ synchronized method: lock on current instance (this) for instance methods, class for static.
- ▶ synchronized block: lock on specified object; finer-grained control.
- ▶ Only one thread can execute synchronized code on the same object at a time.
- ▶ Other threads can execute non-synchronized methods concurrently.
- ▶ Disadvantage: increased waiting time; potential performance degradation.

```
// Synchronized method
synchronized void increment() { count++; }

// Synchronized block (preferred)
void increment() {
    synchronized(this) { count++; }
}
```

wait() / notify() / notifyAll()

Method	Effect	Lock Released?
wait()	Releases lock; enters waiting state until notified	Yes
wait(millis)	Releases lock; waits at most millis ms	Yes
notify()	Wakes ONE random waiting thread	No (released when sync block exits)
notifyAll()	Wakes ALL waiting threads	No

- ▶ All three are in Object class (not Thread).
- ▶ Must be called from synchronized context; else IllegalMonitorStateException.
- ▶ Thread calling wait() must hold the lock on the object.

ReentrantLock (java.util.concurrent.locks)

Method	Description
<code>lock()</code>	Acquire lock (blocks if unavailable)
<code>tryLock()</code>	Attempt to acquire; returns true/false immediately
<code>tryLock(time, TimeUnit)</code>	Wait up to time for lock; returns boolean
<code>lockInterruptibly()</code>	Acquire lock; throws <code>InterruptedException</code> if thread interrupted
<code>unlock()</code>	Release lock (always in finally block!)
<code>isLocked()</code>	Check if lock is held by any thread
<code>isHeldByCurrentThread()</code>	Check if current thread holds the lock
<code>getHoldCount()</code>	Count of re-entrant acquisitions by current thread
<code>getQueueLength()</code>	Number of threads waiting for lock
<code>isFair()</code>	True if fairness policy (longest-waiter first)

Executor Framework

```

ExecutorService service = Executors.newFixedThreadPool(3);
service.submit() -> { /* task */ };           // Runnable
Future<Integer> f = service.submit(callable); // Callable with result
Integer result = f.get();                     // Block until done
service.shutdown(); // Graceful shutdown
service.shutdownNow(); // Immediate shutdown

```

Deadlock

- Occurs when two+ threads wait for each other's locks indefinitely.
- Prevention: consistent lock ordering; use `tryLock()` with timeout; minimize synchronized scope.
- Starvation: thread waits long time but eventually runs (vs deadlock: never runs).

Interview Cheatsheet — Multithreading

Concept / Method	Quick Note
<code>new Thread(r).start()</code>	Start new thread with Runnable r
<code>synchronized(obj)</code>	Lock on obj; only one thread at a time
volatile keyword	Always read from main memory; no caching
<code>AtomicInteger.incrementAndGet()</code>	Thread-safe increment without synchronized
<code>CountDownLatch(n)</code>	Wait for n threads to count down to 0
<code>CyclicBarrier(n)</code>	Wait for n threads to reach barrier point
<code>Semaphore(n)</code>	Allow n threads to access resource simultaneously
<code>ExecutorService.invokeAll(tasks)</code>	Run all Callable tasks; get all Futures
<code>CompletableFuture.supplyAsync()</code>	Async task with result (Java 8+)

Concept / Method	Quick Note
<code>ThreadLocal<T></code>	Per-thread variables; no synchronization needed
<code>ConcurrentHashMap.putIfAbsent(k,v)</code>	Atomic put-if-absent
<code>BlockingQueue.put(e)</code>	Block until space available
<code>BlockingQueue.take()</code>	Block until element available
<code>Thread.sleep()</code> vs <code>wait()</code>	sleep: holds lock; wait: releases lock

12. Inner Classes

Types of Inner Classes

Type	Description	Access Outer Members	Static Members
Member Inner Class	Non-static class inside outer class	Both static & non-static	Not allowed (except static final)
Static Nested Class	Static class inside outer class	Only static members	Allowed (can have main())
Local Inner Class	Class defined inside method/block	Instance members of outer; final/effectively final locals	Not allowed
Anonymous Inner Class	Nameless class defined and instantiated together	Same as local inner	Not allowed

Member Inner Class

```
class Outer {
    int x = 10;
    class Inner {
        void show() { System.out.println(x); } // accesses outer x
    }
}
// Creating instance:
Outer o = new Outer();
Outer.Inner i = o.new Inner();
i.show();
```

- ▶ this refers to inner class; Outer.this refers to outer instance.
- ▶ Applicable modifiers: public, protected, private, default, static, abstract, final, strictfp.

Static Nested Class

```
class Outer {
    static int y = 20;
    static class Inner {
        void show() { System.out.println(y); } // only static
        public static void main(String[] args) { } // can have main
    }
}
// Creating instance:
Outer.Inner i = new Outer.Inner();
```

Anonymous Inner Class

```
// Extending a class
Animal a = new Animal() {
    void sound() { System.out.println("Anonymous sound"); }
};
```

```
// Implementing an interface
Runnable r = new Runnable() {
    public void run() { /* logic */ }
};

// Lambda equivalent (Java 8):
Runnable r2 = () -> { /* logic */ };
```

- ▶ Cannot define constructors (no name).
- ▶ Can extend ONE class OR implement ONE interface (not both).
- ▶ Used for one-time use implementations.

Anonymous Inner Class vs Lambda

Feature	Anonymous Inner Class	Lambda Expression
Interface target	Any number of methods	Only functional interface (1 abstract)
Class extension	Can extend class	Cannot extend class
Instance variables	Can declare	Cannot declare
this keyword	Refers to inner class	Refers to enclosing class
.class file generated	Yes (ClassName\$1.class)	No
Memory	Heap (on demand)	Method area (permanent)

Interview Cheatsheet — Inner Classes & Lambdas

Pattern	Code	Use Case
Anonymous Runnable	<code>new Thread(() -> { /*...*/ }).start();</code>	Quick thread creation
Comparator lambda	<code>list.sort((a,b) -> a.compareTo(b));</code>	Custom sort
Method reference	<code>list.sort(String::compareTo);</code>	Cleaner comparator
Predicate lambda	<code>list.stream().filter(s -> s.startsWith("A"))</code>	Filtering
Function lambda	<code>list.stream().map(String::length)</code>	Transformation
Event listener	<code>button.addActionListener(e -> doAction());</code>	UI events
Callable	<code>Callable<Integer> c = () -> compute();</code>	Task with return value

13. Generics

Why Generics?

- ▶ Type Safety: compile-time type checking prevents `ClassCastException` at runtime.
- ▶ Eliminates explicit type casting when retrieving from collections.
- ▶ Introduced in Java 5.
- ▶ Type parameters must be reference types — no primitives (use wrapper classes).

Generic Class & Methods

```
// Generic class
class Container<T> {
    private T value;
    Container(T value) { this.value = value; }
    T getValue() { return value; }
}

// Generic method
public <T> void print(T obj) { System.out.println(obj); }
```

Wildcards

Wildcard	Syntax	Can Add?	Description
Unbounded	<code>List<?></code>	Only null	List of any type; read-only use
Upper bounded	<code>List<? extends T></code>	Only null	List of T or subtype; for reading
Lower bounded	<code>List<? super T></code>	T or subtype	List of T or supertype; for writing

Bounded Types

```
// Upper bound: T must be Number or subtype
class Stats<T extends Number> { }

// Multiple bounds: class first, then interfaces
class Processor<T extends Number & Runnable & Comparable<T>> { }
```

- ▶ At most one class in multiple bounds; it must come first.
- ▶ Use `extends` keyword for both class and interface bounds.
- ▶ `super` keyword only used in wildcard lower bounds.

Type Erasure

- ▶ Generic type information is REMOVED at compile time (type erasure).
- ▶ JVM sees raw types at runtime: `ArrayList<String>` → `ArrayList`.

- ▶ Cannot overload with same erasure: `void m(List<String>)` and `void m(List<Integer>)` → CE.
- ▶ Arrays and generics don't mix well due to type erasure.

java.util.function Package (Java 8)

Interface	Method	Signature	Description
Predicate<T>	test()	$T \rightarrow \text{boolean}$	Conditional check; used in filter()
Function<T,R>	apply()	$T \rightarrow R$	Transform input to output; used in map()
Consumer<T>	accept()	$T \rightarrow \text{void}$	Consume input; used in forEach()
Supplier<T>	get()	$() \rightarrow T$	Supply a value; used in lazy eval
BiFunction<T,U,R>	apply()	$(T,U) \rightarrow R$	Two inputs, one output
BiPredicate<T,U>	test()	$(T,U) \rightarrow \text{boolean}$	Two inputs, boolean output
UnaryOperator<T>	apply()	$T \rightarrow T$	Function where input = output type
BinaryOperator<T>	apply()	$(T,T) \rightarrow T$	Two same-type inputs, same output

Interview Cheatsheet — Generics & Lambdas

Pattern / Method	Description
Predicate.and() / or() / negate()	Compose predicates: p1.and(p2)
Function.andThen(f)	Compose functions: f1.andThen(f2) applies f1 then f2
Function.compose(f)	f1.compose(f2) applies f2 then f1
Comparator.comparing(fn)	Natural comparison by field: Comparator.comparing(Person::getName)
Comparator.reversed()	Reverse order of comparator
Optional.of(val)	Non-null value wrapper
Optional.ofNullable(val)	Nullable wrapper
Optional.orElse(default)	Return value or default
Optional.orElseThrow()	Return value or throw NoSuchElementException
Optional.map(fn)	Transform value if present
Optional.filter(pred)	Keep value if predicate matches
Collections.emptyList()	Type-safe empty list: Collections.<String>emptyList()

14. Java 8 — Streams & Functional Features

Stream API

Streams provide a declarative way to process collections. They are lazy, functional, and can be parallel.

Operation Type	Examples	Returns
Intermediate (lazy)	<code>filter()</code> , <code>map()</code> , <code>flatMap()</code> , <code>sorted()</code> , <code>distinct()</code> , <code>limit()</code> , <code>skip()</code>	<code>Stream<T></code>
Terminal (eager)	<code>forEach()</code> , <code>collect()</code> , <code>count()</code> , <code>min()</code> , <code>max()</code> , <code>findFirst()</code> , <code>anyMatch()</code>	Result / void
Short-circuit	<code>findFirst()</code> , <code>findAny()</code> , <code>anyMatch()</code> , <code>allMatch()</code> , <code>noneMatch()</code>	May not process all elements

Stream Operations Cheatsheet

Method	Signature	Description
<code>filter()</code>	<code>Predicate</code> → <code>Stream</code>	Keep elements matching predicate
<code>map()</code>	<code>Function</code> → <code>Stream</code>	Transform each element
<code>flatMap()</code>	<code>Function</code> → <code>Stream<Stream></code> → <code>Stream</code>	Flatten nested streams
<code>sorted()</code>	<code>Comparator</code> (opt.) → <code>Stream</code>	Sort elements
<code>distinct()</code>	→ <code>Stream</code>	Remove duplicates
<code>limit(n)</code>	<code>long</code> → <code>Stream</code>	Keep first n elements
<code>skip(n)</code>	<code>long</code> → <code>Stream</code>	Skip first n elements
<code>peek()</code>	<code>Consumer</code> → <code>Stream</code>	Debug; non-terminal side effect
<code>forEach()</code>	<code>Consumer</code> → void	Process each element
<code>collect()</code>	<code>Collector</code> → result	Collect into collection/string/map
<code>count()</code>	→ <code>long</code>	Count elements
<code>min()</code> / <code>max()</code>	<code>Comparator</code> → <code>Optional</code>	Find min/max
<code>findFirst()</code>	→ <code>Optional</code>	First element
<code>anyMatch()</code>	<code>Predicate</code> → boolean	Any element matches?
<code>allMatch()</code>	<code>Predicate</code> → boolean	All elements match?
<code>noneMatch()</code>	<code>Predicate</code> → boolean	No elements match?
<code>reduce()</code>	<code>BinaryOperator</code> → <code>Optional</code>	Combine elements to single value
<code>mapToInt()</code>	<code>Function</code> → <code>IntStream</code>	Convert to <code>IntStream</code> (sum, avg, etc.)
<code>toArray()</code>	→ <code>Object[]</code>	Collect to array

Collectors

Collector	Description
<code>Collectors.toList()</code>	Collect to ArrayList
<code>Collectors.toSet()</code>	Collect to HashSet
<code>Collectors.toMap(kFn, vFn)</code>	Collect to HashMap
<code>Collectors.joining(delim)</code>	Concatenate strings with delimiter
<code>Collectors.groupingBy(fn)</code>	Group by function: <code>Map<K, List<T>></code>
<code>Collectors.partitioningBy(pred)</code>	Partition into true/false groups: <code>Map<Boolean, List<T>></code>
<code>Collectors.counting()</code>	Count elements in each group
<code>Collectors.summarizingInt(fn)</code>	Stats: count, sum, min, max, avg
<code>Collectors.averagingInt(fn)</code>	Average of int values
<code>Collectors.toUnmodifiableList()</code>	Immutable list (Java 10+)

Method References

Type	Syntax	Equivalent Lambda
Static method	<code>ClassName::staticMethod</code>	<code>x -> ClassName.staticMethod(x)</code>
Instance method (specific)	<code>obj::instanceMethod</code>	<code>x -> obj.instanceMethod(x)</code>
Instance method (arbitrary)	<code>ClassName::instanceMethod</code>	<code>(o,x) -> o.instanceMethod(x)</code>
Constructor	<code>ClassName::new</code>	<code>args -> new ClassName(args)</code>

Common Stream Patterns

```
// Sum of list
int sum = list.stream().mapToInt(Integer::intValue).sum();

// Group by length
Map<Integer, List<String>> grouped = list.stream()
    .collect(Collectors.groupingBy(String::length));

// Find max element
Optional<Integer> max = list.stream().max(Comparator.naturalOrder());

// Parallel stream
list.parallelStream().filter(n -> n > 0).count();

// Collect to sorted set
Set<String> sorted = list.stream().collect(Collectors.toCollection(TreeSet::new));
```

Common Pattern	Stream Solution
Sum of integers	<code>list.stream().mapToInt(i -> i).sum()</code>
Average	<code>list.stream().mapToDouble(i -> i).average().orElse(0)</code>
Max / Min	<code>list.stream().max(Comparator.naturalOrder()).get()</code>
Filter + collect	<code>list.stream().filter(pred).collect(Collectors.toList())</code>
Word frequency	<code>Arrays.stream(words).collect(Collectors.groupingBy(w->w, Collectors.counting()))</code>
Flatten list of lists	<code>lists.stream().flatMap(Collection::stream).collect(Collectors.toList())</code>
Sort by field	<code>list.stream().sorted(Comparator.comparing(Person::getAge))</code>
Remove duplicates	<code>list.stream().distinct().collect(Collectors.toList())</code>
String joining	<code>list.stream().collect(Collectors.joining(", "))</code>
Partition by predicate	<code>list.stream().collect(Collectors.partitioningBy(pred))</code>
Count by group	<code>list.stream().collect(Collectors.groupingBy(fn, Collectors.counting()))</code>
Check all match	<code>list.stream().allMatch(pred)</code>
Optional chain	<code>Optional.ofNullable(s).map(String::length).orElse(0)</code>

15. Garbage Collection & JVM

Garbage Collection Basics

- ▶ GC automatically destroys unreferenced (unreachable) objects.
- ▶ GC is a low-priority daemon thread managed by JVM.
- ▶ Programmer is only responsible for creating objects; GC handles destruction.
- ▶ Most GC implementations follow Mark-and-Sweep algorithm.

Making Objects GC-Eligible

Method	Example
Nullify reference	<code>obj = null;</code>
Reassign reference	<code>obj = new OtherObject();</code>
Local variable out of scope	Method returns; local object unreferenced
Island of isolation	Objects reference each other but no external reference exists

Requesting GC

```
System.gc(); // Static method hint
Runtime.getRuntime().gc(); // Instance method hint
// Neither guarantees immediate collection
```

finalize() Method

- ▶ Called by GC before destroying an object (deprecated since Java 9).
- ▶ `finalize()` is called only ONCE per object, even if eligible for GC multiple times.
- ▶ If `finalize()` throws exception, it is ignored.
- ▶ Explicit call to `finalize()` = normal method call; object not destroyed.
- ▶ Prefer try-with-resources and `AutoCloseable` for resource cleanup.

JVM Execution Flow

Step	Action
1. Check <code>main()</code>	JVM verifies <code>main()</code> exists (Java 7+); throws error if missing
2. Load class	<code>ClassLoader</code> loads <code>.class</code> file into method area
3. Initialize static	Static variables assigned defaults; static blocks execute in order

Step	Action
4. Execute <code>main()</code>	<code>main()</code> thread starts execution
5. GC runs	Daemon GC thread reclaims unreferenced objects during execution
6. JVM shutdown	When <code>main()</code> and all non-daemon threads complete

Memory Areas in JVM

Area	Contents
Method Area (Metaspace)	Class metadata, static variables, static blocks, constant pool
Heap	Instance objects, String Constant Pool (Java 7+)
Stack	Method frames, local variables, partial results
PC Register	Current instruction pointer per thread
Native Stack	Native method calls

Interview Cheatsheet — GC & JVM

Concept / Method	Description
<code>System.gc()</code>	Request GC (not guaranteed)
<code>Runtime.getRuntime().totalMemory()</code>	Total JVM heap memory
<code>Runtime.getRuntime().freeMemory()</code>	Free heap memory
<code>Runtime.getRuntime().maxMemory()</code>	Maximum memory JVM can use
<code>WeakReference<T></code>	GC can collect even if weakly referenced
<code>SoftReference<T></code>	GC collects only if memory low
<code>PhantomReference<T></code>	Post-finalization cleanup
<code>java.lang.ref.Cleaner</code> (Java 9+)	Safe alternative to <code>finalize()</code>
<code>-Xms256m -Xmx512m</code>	JVM flags: initial and max heap size
<code>System.exit(0)</code>	Terminates JVM immediately; prevents finally